

# Vizsgajegyzet: Support Vector Machines

ML-005 Lecture 12 | Stanford University

2025. június 14., 16:50 CEST

## Forrás

Diasor („svm.pptx”) és ML-005 Lecture 12 eladás

## 1. Bevezetés

A Support Vector Machines (SVM) egy hatékony gépi tanulási algoritmus, amelyet elsősorban osztályozási, de esetenként regressziós feladatok megoldására használnak. Az SVM célja egy olyan döntési határ (hiperlap) megtalálása, amely maximális margóval választja el a különböző osztályokat, így biztosítva a legjobb általánosítási képességet. Az SVM különösen hatékony lineárisan elválasztható adatok esetén, de a kernel trükk alkalmazásával nemlineáris problémákra is kiterjeszthet. Alkalmazási területei közé tartozik a képfelismerés, szövegminsités, bioinformatika és pénzügyi előrejelzések.

- **Cél:** Osztályozási problémák megoldása maximális margójú döntési határral.
- **Példák:**
  - Kézírásos számjegyek osztályozása (pl. MNIST adathalmaz).
  - Spam e-mailek szűrése.
  - Daganatok jóindulatú/rosszindulatú osztályozása orvosi adatok alapján.
- **Tanulási típus:** Felügyelt tanulás, címkézett adatokkal:  $(x^{(i)}, y^{(i)})$ , ahol  $y^{(i)} \in \{1, -1\}$  bináris osztályozás esetén.
- **Kontextus:** Az SVM optimális döntési határt keres, amely robusztus a zajjal szemben és jól általánosít új adatokra.

## 2. Lineáris SVM

A lineáris SVM egy bináris osztályozási probléma megoldására szolgál, ahol az adatok lineárisan elválaszthatók. A cél egy olyan hiperlap megtalálása, amely maximális margóval választja el a pozitív ( $y = 1$ ) és negatív ( $y = -1$ ) osztályokat.

- **Adathalmaz:** Tanító példák:  $(x^{(i)}, y^{(i)})$ , ahol  $i = 1, \dots, m$ ,  $x^{(i)} \in \mathbb{R}^n$ ,  $y^{(i)} \in \{1, -1\}$ .

- **Hiperlap:** A döntési határ egy hiperlap, amelyet a következő egyenlet definiál:  $\mathbf{w}^T \mathbf{x} + b = 0$ , ahol  $\mathbf{w}$  a súlyvektor,  $b$  az eltolás.
- **Margó:** A margó a hiperlap és a legközelebbi adatpontok (támaszvektorok) közötti távolság, amelyet  $\frac{2}{\|\mathbf{w}\|}$ -ként definiálnak.
- **Kemény margós SVM** (Hard-margin SVM, tökéletesen elválasztható adatok):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

tárgyalva:

$$y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \forall i$$

Ez maximalizálja a margót ( $\frac{2}{\|\mathbf{w}\|}$ ) azáltal, hogy minimalizálja  $\|\mathbf{w}\|^2$ -t.

- **Puha margós SVM** (Soft-margin SVM, nem tökéletesen elválasztható adatok): Bevezetjük a lazító változókat ( $\xi_i$ ) a hibák kezelésére:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

tárgyalva:

$$y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

- $C$ : Regularizációs paraméter, amely szabályozza a margó maximalizálása és a hibák minimalizálása közötti egyensúlyt.
- Nagy  $C$ : Kevésbé tolerálja a hibákat, szigorúbb margó.
- Kis  $C$ : Több hibát engedélyez, nagyobb margó.
- **Döntési függvény:** Az osztályozás a következőképpen történik:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

- **Támaszvektorok:** Azok az adatpontok, amelyek a margó szélén helyezkednek el ( $y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) = 1$ ) vagy a margón belül vannak ( $\xi_i > 0$ ).

### 3. Dual Probléma és Lagrange Szorzók

Az SVM optimalizálási problémáját gyakran a dual formában oldják meg, amely hatékonyabb számításokat tesz lehetővé, különösen kernel-ekkel.

- **Dual probléma:**

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} (\mathbf{x}^{(i)})^T \mathbf{x}^{(j)}$$

tárgyalva:

$$\sum_{i=1}^m \alpha_i y^{(i)} = 0, \quad 0 \leq \alpha_i \leq C \quad \forall i$$

- $\alpha_i$ : Lagrange szorzók, amelyek a támaszvektorokhoz tartoznak ( $\alpha_i > 0$ ).
- A nem támaszvektorokhoz  $\alpha_i = 0$ .

- **Súlyvektor és eltolás:**

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y^{(i)} \mathbf{x}^{(i)}$$

Az eltolás ( $b$ ) kiszámítható egy támaszvektorból:

$$b = y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} \quad (\text{bármely támaszvektorra})$$

- **Döntési függvény:**

$$h(\mathbf{x}) = \text{sign} \left( \sum_{i=1}^m \alpha_i y^{(i)} (\mathbf{x}^{(i)})^T \mathbf{x} + b \right)$$

## 4. Nemlineáris SVM és Kernel Trükk

Ha az adatok nem lineárisan elválaszthatók az eredeti térben, a kernel trükk segítségével magasabb dimenziós térbe transzformáljuk ket, ahol lineárisan elválaszthatóak lehetnek.

- **Kernel függvény:**  $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}')$ , ahol  $\phi$  a transzformáció a magasabb dimenziós térbe. A kernel trükk lehetővé teszi, hogy ne kelljen explicit módon kiszámítani  $\phi(\mathbf{x})$ -et.
- **Dual probléma kernel-ekkel:**

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

tárgyalva:

$$\sum_{i=1}^m \alpha_i y^{(i)} = 0, \quad 0 \leq \alpha_i \leq C \quad \forall i$$

- **Döntési függvény:**

$$h(\mathbf{x}) = \text{sign} \left( \sum_{i=1}^m \alpha_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}) + b \right)$$

- **Gyakori kernel függvények:**

- **Lineáris kernel:**  $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$ .
- **Polinomiális kernel:**  $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d$ , ahol  $c \geq 0$ ,  $d$  a fokszám.
- **Gaussian (RBF) kernel:**

$$K(\mathbf{x}, \mathbf{x}') = \exp \left( -\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2} \right)$$

- **Sigmoid kernel:**  $K(\mathbf{x}, \mathbf{x}') = \tanh(\beta \mathbf{x}^T \mathbf{x}' + \gamma)$ .

- **Példa Gaussian kernel-re:** Ha  $\mathbf{x} = [1, 2]^T$ ,  $\mathbf{x}' = [2, 3]^T$ ,  $\sigma^2 = 1$ :

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{(1-2)^2 + (2-3)^2}{2 \cdot 1}\right) = \exp(-1) \approx 0.3679$$

- **Landmarkok:** A Gaussian kernel esetén minden tanítópont ( $\mathbf{x}^{(i)}$ ) egy landmark ( $l^{(i)}$ ), és a hasonlósági függvény:

$$f_i = \exp\left(-\frac{\|\mathbf{x} - l^{(i)}\|^2}{2\sigma^2}\right)$$

## 5. Paraméterek és Hatásuk

Az SVM teljesítménye nagymértékben függ a paraméterek megfelelő beállításától.

- **$C$  paraméter:**
  - Nagy  $C$ : Kevésbé tolerálja a hibákat, szigorúbb margó, alacsonyabb bias, de magasabb variancia (túlillesztés kockázata).
  - Kis  $C$ : Több hibát engedélyez, nagyobb margó, magasabb bias, de alacsonyabb variancia (alulillesztés kockázata).
- **Kernel paraméterek:**
  - Gaussian kernel:  $\sigma^2$  szabályozza a hasonlósági függvény szélességét.
    - \* Nagy  $\sigma^2$ : Simább döntési határ, alacsonyabb variancia, de magasabb bias.
    - \* Kis  $\sigma^2$ : Élesebb döntési határ, alacsonyabb bias, de magasabb variancia.
  - Polinomiális kernel:  $d$  (fokszám) és  $c$  (eltolás) befolyásolja a döntési határ komplexitását.
- **Példa táblázat:**

| Paraméter             | Nagy érték                   | Kis érték                    |
|-----------------------|------------------------------|------------------------------|
| $C$                   | Szigorú margó, alacsony bias | Nagy margó, magas bias       |
| $\sigma^2$ (Gaussian) | Simább határ, magas bias     | Élesebb határ, alacsony bias |

1. táblázat. Paraméterek hatása az SVM teljesítményére.

## 6. Praktikus Megfontolások

- **Kernel kiválasztása:** A kernel típusának megválasztása az adatok természetétől függ. Lineáris kernel egyszerűbb problémákhoz, Gaussian kernel komplex, nemlineáris problémákhoz ajánlott.
- **Paraméter hangolás:** Használjunk rácskeresést (grid search) vagy véletlenszerű keresést a legjobb  $C$  és kernel paraméterek (pl.  $\sigma^2$ ) megtalálására. Keresztvalidációval ellenrizzük a teljesítményt.

- **Jellemz skálázás:** Az SVM érzékeny a jellemz skálájára, ezért normalizáljuk az adatokat (pl.  $[-1, 1]$  vagy  $[0, 1]$  intervallumra).
- **Szoftverek:** Az SVM implementálása számításigényes, ezért használjunk bevált könyvtárakat:
  - **Libsvm:** Általános célú SVM könyvtár ([Libsvm](https://www.csie.ntu.edu.tw/~cjlin/libsvm/)).
  - **Liblinear:** Lineáris SVM-ekhez optimalizált ([Liblinear](https://www.csie.ntu.edu.tw/~cjlin/liblinear/)).
  - **Scikit-learn:** Python alapú, könnyen használható ([Scikit-learn](https://scikit-learn.org/stable/modules/svm.html)).
- **Kiindulási pontok:** Kezdjük lineáris kernellel, majd próbáljuk ki a Gaussian kernelt, ha szükséges.  $C$  és  $\sigma^2$  kezdeti értékei:  $C = 1$ ,  $\sigma^2 = \frac{1}{\text{jellemz száma}}$ .

## 7. Összehasonlítás Logisztikus Regresszióval

Az SVM és a logisztikus regresszió egyaránt bináris osztályozási problémák megoldására alkalmas, de eltér megközelítést alkalmaznak.

- **Logisztikus regresszió:**
  - **Hipotézis:**  $h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}$ , amely valószínűségeket ad ( $P(y = 1|x)$ ).
  - **Költségfüggvény:** Log-loss:
 
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$
  - **Elny:** Robusztus zajos adatokkal, valószínűségi kimenet.
  - **Hátrány:** Nem optimalizálja a margót, kevésbé hatékony jól elválasztható adatok esetén.
- **SVM:**
  - **Hipotézis:**  $\text{sign}(\mathbf{w}^T \mathbf{x} + b)$ , amely csak osztálycímkeket ad.
  - **Költségfüggvény:** Hinge loss (puha margós SVM-ben):
 
$$J(\mathbf{w}, b, \xi) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$
  - **Elny:** Maximális margó, jobb általánosítás jól elválasztható adatokon.
  - **Hátrány:** Nem ad valószínűségeket, számításigényesebb.
- **Különbségek táblázata:**

| Tulajdonság     | Logisztikus Regresszió | SVM               |
|-----------------|------------------------|-------------------|
| Kimenet         | Valószínűség           | Osztálycímke      |
| Költségfüggvény | Log-loss               | Hinge loss        |
| Margó           | Nem optimalizálja      | Maximalizálja     |
| Zajérzékenység  | Robusztusabb           | Kevésbé robusztus |

2. táblázat. Logisztikus regresszió és SVM összehasonlítása.

## 8. Példa Alkalmazás

- **Probléma:** Kézírásos számjegyek osztályozása (MNIST adathalmaz, 0 vs. 1).
- **Adatok:**  $28 \times 28$  pixeles képek,  $x^{(i)} \in \mathbb{R}^{400}$ ,  $y^{(i)} \in \{1, -1\}$ .
- **Folyamat:**
  1. **Adat elkészítés:** Normalizáljuk a pixelértékeket  $[0, 1]$ -re.
  2. **Kernel választás:** Próbáljuk ki a Gaussian kernelt ( $\sigma^2 = 0.1$ ).
  3. **Paraméterek:**  $C = 1$ , rácskereséssel finomhangoljuk.
  4. **Tanítás:** Használjuk a scikit-learn SVC osztályát: `[language=Python] from sklearn.svm import SVC model = SVC(kernel='rbf', C=1, gamma=0.1) model.fit(X_train, y_train)`
  5. **Predikció:** Új képre:

$$h(\mathbf{x}) = \text{sign} \left( \sum_{i=1}^m \alpha_i y^{(i)} \exp \left( -\frac{\|\mathbf{x}^{(i)} - \mathbf{x}\|^2}{2 \cdot 0.1} \right) + b \right)$$

- **Eredmény:** Pl. 98% pontosság a validációs adathalmazon.

## 9. Kulcsformulák

- **Lineáris SVM primal:**

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

tárgyalva:

$$y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i$$

- **Dual probléma:**

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

tárgyalva:

$$\sum_{i=1}^m \alpha_i y^{(i)} = 0, \quad 0 \leq \alpha_i \leq C \quad \forall i$$

- **Kernel SVM döntési függvény:**

$$h(\mathbf{x}) = \text{sign} \left( \sum_{i=1}^m \alpha_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}) + b \right)$$

- **Gaussian kernel:**

$$K(\mathbf{x}, \mathbf{x}') = \exp \left( -\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2} \right)$$